

Hugging Face ile Model Fine-Tuning

Temelden Uygulamaya Kapsamlı Rehber

Güncel: Temmuz 2026 — TRL v1.x API'si

Bölüm 1 — Temel Kavramlar

1.1 Fine-tuning nedir, ne değildir?

Büyük dil modelleri (LLM) iki aşamada oluşur:

- Pretraining (ön eğitim):** Model, internet ölçeğinde metinle "bir sonraki token'ı tahmin et" görevinde eğitilir. Dilin genel yapısını ve dünya bilgisini burada öğrenir. Milyonlarca dolar ve binlerce GPU gerektirir — bunu siz yapmazsınız.
- Fine-tuning (ince ayar):** Halihazırda eğitilmiş bir modeli, kendi göreve özel küçük verinizle kısaca eğitip davranışını uyarlıyorsunuz. Bu **transfer learning**'in bir biçimidir: genel bilgiyi koru, spesifik beceriyi ekle.

Kritik ayrım — fine-tuning ne için değildir:

- Yeni bilgi yüklemek için zayıftır. "Şirketimin 2026 fiyat listesini modele öğret" → yanlış araç. Bunun için **RAG** (retrieval-augmented generation) kullanın.
- Davranış, format, ton, stil ve görev uzmanlığı için doğru araçtır. "Her zaman JSON döndür", "hukuki dilde özetle", "bu sınıflandırma görevini yap".

Zihinsel kural: Fine-tuning modele *nasıl davranacağını* öğretir, *ne bildiğini* değil.

1.2 Token ve tokenization

Model metni doğrudan görmez; **token** denen parçalara böler (bir kelime, kelime parçası veya karakter). "fine-tuning" yaklaşık 3-4 token olabilir. Tokenizer bu dönüşümü yapan bileşendir ve **her model kendi tokenizer'ıyla gelir** — modeli ve tokenizer'ı asla karıştırmayın.

`max_length` = modele tek seferde verilen maksimum token sayısı. Uzun tutmak bellek maliyetini artırır.

1.3 Eğitim döngüsünün anatomisi

Kavram	Anlamı	Sezgi
Epoch	Tüm verinin bir kez baştan sona geçmesi	Fazlası ezberlemeye (overfit) yol açar; genelde 1–3
Batch size	Aynı anda işlenen örnek sayısı	Büyük = stabil ama VRAM yer
Gradient accumulation	Küçük batch'leri toplayıp büyük batch etkisi yaratma	VRAM'e sığdırma hilesi
Learning rate (LR)	Her adımda ağırlıkların ne kadar güncellendiği	En kritik hiperparametre
Loss	Modelin tahmin hatası	Eğitimde düşmeli; val'de yükselirse dur
Step	Bir batch üzerinde tek güncelleme	—

Efektif batch = batch_size × gradient_accumulation × GPU sayısı.

1.4 Overfitting (ezberleme) ve doğrulama

Model eğitim verisini "ezberleyip" yeni veride başarısız olabilir. Korunma:

- Veriyi **train / validation / test** olarak ayırın.
- Eğitim loss'u düşerken **validation loss yükseliyorsa** → overfit, durun.
- **Data leakage**: Test örneği yanlışlıkla train'e sızarsa metrikler yalancı şekilde iyi çıkar — kontrol edin.
- **n=1 kanıt değildir**: Birkaç örnekte "güzel cevap verdi" demek başarı ölçümü değildir; sistematik metrik + baseline şarttır.

1.5 Full fine-tuning'in problemi ve PEFT çözümü

7 milyar parametrelili bir modelin **tüm** ağırlıklarını eğitmek: her parametrenin kendisi + gradyanı + optimizier state'i bellekte tutulur → onlarca GB VRAM. Çoğu kişi için imkânsız.

PEFT (Parameter-Efficient Fine-Tuning): Modelin %99'unu dondur, sadece küçük bir eklenti eğit. Kalite neredeyse full FT kadar, maliyet çok düşük.

1.6 LoRA — sezgisel açıklama

LoRA (Low-Rank Adaptation): Büyük bir ağırlık matrisi W 'yi doğrudan değiştirmek yerine, yanına **iki küçük matris (A ve B)** ekler. Güncelleme $W + A \cdot B$ olarak temsil edilir; A ve B çok düşük "rank"lı (ince) olduğundan toplam eğitilebilir parametre %90+ azalır.

- **r (rank)**: Adapter'ın "kalınlığı" / kapasitesi. Yüksek **r** = daha çok kapasite ama overfit riski. Tipik: 8–64.
- **lora_alpha**: Ölçekleme katsayısı; pratikte $\alpha \approx 2r$.
- **target_modules**: Adapter'ın hangi katmanlara ekleneceği. 2026 tavsiyesi: **"all-linear"** (tüm linear katmanlar).

Eğitim sonunda elde ettiğiniz adapter dosyası sadece birkaç MB — base modeli değiştirmeden yüklenir/çıkarılır.

1.7 Quantization ve QLoRA

Quantization: Ağırlıkları daha az bit'le saklamak (ör. 16-bit → **4-bit**). Bellek yaklaşık 4x azalır, küçük bir kalite bedeliyle.

QLoRA = 4-bit quantize edilmiş donmuş base + üstüne LoRA adapter. Böylece tek bir GPU'da 7B–70B modeli fine-tune edebilirsiniz. 2026'da pratik varsayılan yöntem budur.

- **NF4 (normal-float 4):** QLoRA için tasarlanmış 4-bit veri tipi.
- **Double quantization:** Quantization sabitlerini de sıkıştırıp ekstra bellek kazancı.

1.8 SFT ve sonrası — hizalama merdiveni

- **SFT (Supervised Fine-Tuning):** "girdi → doğru çıktı" örnekleriyle eğitim. Fine-tuning'in temel biçimi.
- **Preference tuning (DPO / GRPO / KTO):** SFT sonrası, "bu cevap şundan daha iyi" tercih sinyaliyle modeli hizalama (ton, yardımcılık, güvenlik). RLHF'in modern, basitleşmiş halleri.

Sıralama: **Pretrain** → **SFT** → (**opsiyonel**) **Preference tuning**.

Bölüm 2 — Ekosistem ve Güncel Sürüm Değişiklikleri (2026)

Üç ana kütüphane:

- **transformers** — model yükleme, tokenizer, inference.
- **peft** — LoRA/QLoRA implementasyonu.
- **trl** — LLM'e özel eğitim araçları (**SFTTrainer** vb.).

TRL v1.0 (Nisan 2026) birleşik bir "post-training" yığınına geçti (SFT, Reward Modeling, DPO, GRPO tek çatı). Bu, eski tutorial'ları kıran **breaking change**'ler getirdi:

Eski (artık YANLIŞ)	Yeni (v1.x)
<code>SFTTrainer(tokenizer=...)</code>	<code>processing_class=...</code>
<code>SFTConfig(max_seq_length=...)</code>	<code>max_length=...</code>
<code>dataset_text_field</code> , <code>packing</code> trainer argümanı	Artık SFTConfig alanları
Trainer'a düz <code>TrainingArguments</code>	Her trainer'ın kendi Config'i: <code>SFTConfig</code> , <code>DPOConfig</code> , <code>GRPOConfig</code> (hepsi <code>TrainingArguments</code> 'tan türer)

Yaklaşan (TRL 1.7): varsayılan `loss_type` `'nll'` → `'chunked_nll'`. Aynı matematik, daha düşük bellek; standart modellerde şeffaf.

Kurulum (sürümleri pinleyin):

```
pip install "trl>=1.6,<2" "peft>=0.14" "transformers>=4.50" \
"bitsandbytes>=0.43" "datasets>=3.0" accelerate
```

Bölüm 3 — Ücretsiz ve Ücretli Seçenekler (Fiyatlar)

Not: GPU kiralama fiyatları oynaktır. Aşağıdaki rakamlar Temmuz 2026 on-demand göstergesel değerlerdir; işlem öncesi sağlayıcının kendi fiyat sayfasından doğrulayın.

3.1 Ücretsiz seçenekler

Platform	Ne sunar	Sınır / Not
Google Colab (Free)	T4 GPU (16 GB)	Oturum birkaç saat, kesintiye uğrayabilir. 7B QLoRA + küçük veri için yeterli. Öğrenme/prototip.
Kaggle Notebooks	Ücretsiz T4×2 veya P100	Haftalık ~30 saat GPU kotası. Colab'dan daha uzun kesintisiz oturum — en cömert ücretsiz seçenek.
HF AutoTrain (free tier)	No-code eğitim	Sınırlı sample sayısına kadar ücretsiz; üstünde tahmini maliyet gösterilir.
HF Spaces (free CPU)	Demo/inference	Eğitim için değil; çıktı sergileme.

Pratik sonuç: Öğrenmek ve küçük deneyler için **Kaggle free** (daha uzun oturum + haftalık kota) çoğu zaman Colab free'den daha iyidir. 7B modeli QLoRA ile küçük veri üzerinde bu iki platformda **\$0** maliyetle fine-tune edebilirsiniz.

3.2 Ücretli — kendi kodunuz, saatlik GPU kiralama

Temmuz 2026, on-demand, göstergesel:

Sağlayıcı	GPU	~Fiyat/saat	Not
RunPod	RTX 4090	~\$0.44	Per-second faturalama
RunPod	A100 80GB	~\$1.09–1.39	Dengeli, popüler
RunPod	H100	~\$2.39 (PCIe ~\$1.99)	—
Vast.ai	RTX 4090 (spot)	~\$0.25–0.30	En ucuz, kesinti riskli
Vast.ai	A100 PCIe	~\$0.67 (medyan)	Marketplace fiyatı
Lambda	A100 80GB SXM	~\$2.79 / GPU	8'li sistemlerde
Lambda	H100 PCIe	~\$3.29	Büyük/stabil eğitim
Spheron	A100	~\$0.76	—

Spot (kesintili) instance'lar en ucuzudur ama iş yarıda kesilebilir → **sık checkpoint alın**.

3.3 Ücretli — managed / no-code

- HF AutoTrain:** Kullanılan compute-dakikası başına faturalanır (abonelik primi yok). Küçük model kısa run → birkaç dolar; 70B uzun run → yüzlerce dolar. Eğitim öncesi **tahmini maliyet gösterilir**, onaylayınca başlar.

- **HF Train on DGX Cloud:** H100 **\$8.25 / GPU-saat**, L40S **\$2.75 / GPU-saat**.
- **HF abonelik katmanları:** Free / **PRO \$9/ay** / Team & Enterprise. Dikkat: abonelik ≠ compute; eğitim compute'u ayrıca faturalanır.

3.4 Somut görev maliyetleri (QLoRA, göstergesel)

Görev	Donanım / süre	Yaklaşık maliyet
7B QLoRA, küçük veri	Colab/Kaggle free	\$0
7B QLoRA	A100, ~2–4 saat	~\$1.5–3 (çoğu durumda < \$5)
70B QLoRA	H100, ~8–12 saat	~\$10–16

3.5 Karar akışı — nereyi seçmeli?

Öğrenme / küçük deney	→ Kaggle free (en cömert) veya Colab free
Ciddi, tekrarlı iş, tam kontrol	→ RunPod (per-second, dengeli)
En ucuz büyük batch	→ Vast.ai spot (checkpoint sık al!)
Büyük, stabil eğitim	→ Lambda / H100
Kod yazmak istemiyorum	→ HF AutoTrain (maliyeti önceden gör)

Bölüm 4 — Uçtan Uca QLoRA SFT Örneği (güncel API)

```
import torch
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig

MODEL_ID = "meta-llama/Llama-3.1-8B-Instruct"

# 1) 4-bit quantization (QLoRA'nin "Q"su)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)

model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, quantization_config=bnb_config, device_map="auto",
)
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token # yaygın tuzak

# 2) LoRA adapter
peft_config = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules="all-linear",
    task_type="CAUSAL_LM",
)

# 3) Veri - chat formati; template otomatik uygulanır
dataset = load_dataset("your/dataset", split="train")
# beklenen yapı: {"messages": [{"role": "user", ...}, {"role": "assistant", ...}]}

# 4) Config - YENİ alan isimlerine dikkat
sft_config = SFTConfig(
    output_dir="./out",
    max_length=2048, # max_seq_length DEĞİL
    packing=True, # kısa örnekleri birleştir, throughput artar
    completion_only_loss=True, # Loss SADECE assistant cevabından
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8, # efektif batch = 16
    learning_rate=2e-4, # PEFT ~10x yüksek (full FT ~2e-5)
    lr_scheduler_type="cosine",
    warmup_ratio=0.03,
    num_train_epochs=3,
    bf16=True,
    gradient_checkpointing=True,
    logging_steps=10,
    save_strategy="epoch",
)

trainer = SFTTrainer(
    model=model,
    args=sft_config,
    train_dataset=dataset,
    processing_class=tokenizer, # tokenizer= DEĞİL
    peft_config=peft_config,
)

trainer.train()
trainer.save_model("./final-adapter") # sadece adapter (~MB)
```

Bölüm 5 — Hiperparametre Rehberi

- **Learning rate:** PEFT'te full FT'ye göre **~10x yüksek** (çünkü çok az parametre eğitiliyor). QLoRA için `1e-4 – 3e-4` iyi başlangıç.
- **LoRA rank (`r`):** Küçük veri → düşük `r` (8–16). `alpha ≈ 2r`.
- `target_modules="all-linear"` : Sadece attention değil tüm linear katmanlar → 2026 tavsiyesi, kaliteyi belirgin artırır.
- `completion_only_loss=True` : Instruction tuning'de neredeyse her zaman istediğiniz şey (prompt'tan loss alma).
- **Efektif batch'i** VRAM'e göre değil, hedef efektif batch'e göre ayarlayın (grad_accum ile).

Bölüm 6 — Doğrulama Disiplini

1. **Ayrır:** train/val/test. Val loss'u izle; yükseliyorsa erken dur.
2. **Baseline:** Fine-tune öncesi base modelin aynı görevdeki skorunu ölç. Karşılaştırma olmadan "iyileşti" iddiası geçersizdir.
3. **Data leakage kontrolü:** Özellikle packing + tekrarlı veride.
4. **Görev metriği:** Loss yetmez — exact-match / ROUGE / task accuracy / LLM-as-judge ekle.

Bölüm 7 — Inference (adapter'ı kullanma)

```
from peft import PeftModel
base = AutoModelForCausalLM.from_pretrained(MODEL_ID, device_map="auto")
model = PeftModel.from_pretrained(base, "./final-adapter")
# Isteğe bağlı: model = model.merge_and_unload() # adapter'ı ağırlıklara göm
```

Not: 4-bit base üstünde `merge_and_unload` kalite kaybı yaratır; merge edecekseniz base'i fp16/bf16 yükleyin.

Bölüm 8 — Hız / Bellek Optimizasyonu

- **Unsloth:** TRL v1'de SFT/DPO'da **~2x hız, ~%70 daha az bellek**. Tek GPU sınırındaysanız değerlendirin.
- `accelerate` / **DeepSpeed** / **FSDP**: Multi-GPU ölçekleme.
- `packing=True` + `gradient_checkpointing` : Neredeyse bedava throughput ve bellek kazancı.

Bölüm 9 — Genel Karar Akışı

```
Görev format/davranış/stil mi? → SFT (QLoRA ile başla)
Cevap tercihi / hizalama mı? → SFT + DPO
Yeni/değişken bilgi mi? → Fine-tuning DEĞİL, RAG
Tek GPU'da büyük model mi? → QLoRA (4-bit) + Unsloth
Her durumda: → baseline ölç, val izle, sürüm pinle,
                güncel API kullan (processing_class,
                max_length, SFTConfig alanları)
```

Kaynaklar

- Hugging Face TRL v1.0 Release — MarkTechPost (Nisan 2026)
- huggingface/trl Releases — GitHub
- trl SFTConfig kaynak kodu — GitHub
- PEFT Integration — TRL Dokümantasyonu
- PEFT — GitHub
- Fine-Tune Gemma with QLoRA — Google AI for Developers
- GPU Cloud Pricing Comparison 2026 (RunPod, Vast.ai, Lambda, Spheron)
- Hugging Face Pricing & AutoTrain / DGX Cloud — huggingface.co/pricing

Fiyat ve sürüm bilgileri Temmuz 2026 itibarıyladır ve değişebilir.